

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 4
По дисциплине «Современные платформы программирования»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
03.04.2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Ход работы:

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

10. Автоматический анализ самых активных контрибьюторов в репозитории

Напишите Python-скрипт, который анализирует самых активных контрибьюторов в указанном GitHub-репозитории за определённый период.

1. Запрашивает у пользователя:

- Репозиторий (owner/repo, например, tensorflow/tensorflow).
- Период анализа (за неделю, за месяц, за год).
- (Опционально) Минимальное количество коммитов для попадания в рейтинг

2. Использовать GitHub API для получения списка контрибьюторов и их активности:

- Количество коммитов.
- Количество добавленных и удалённых строк кода.
- Количество открытых и закрытых pull requests.
- Количество открытых и закрытых issues.
- Количество комментариев к PR и issues.

3. Определить ТОП-5 самых активных контрибьюторов.

4. Построить график активности (matplotlib, seaborn).

```
import datetime
from datetime import datetime, timedelta
import requests
import matplotlib.pyplot as plt
```

```
TOKEN =
```

```
"github_pat_11AQRDVRA03QxtA3EKwnt5_io6fzeGUSXXck0IUw5CxIKOutxQ7Oa3ns2cTXNDDLuT
O7UMWXXwcdcZ7n3"
```

```
def get_top_contributors(repo_name, days_count, min_commits_count):
    since = (datetime.now() - timedelta(days=days_count)).isoformat()
```

```
headers = {"User-Agent": "Mozilla/5.0", "Authorization": f"token {TOKEN}"}
```

```
url = f"https://api.github.com/repos/{repo_name}/contributors"  
response = requests.get(url, headers=headers, timeout=30)
```

```
if response.status_code != 200:  
    return []
```

```
contributors = response.json()  
result = []
```

```
for contributor in contributors[:20]:  
    login = contributor["login"]
```

```
url_commits = f"https://api.github.com/repos/{repo_name}/commits"  
params = {"author": login, "since": since, "per_page": 1}  
r = requests.get(url_commits, headers=headers, params=params, timeout=30)
```

```
if "Link" in r.headers:  
    link = r.headers["Link"]  
    last = link.split("page=")[1].split(">")[0]  
    commits = int("".join(filter(str.isdigit, last)))  
else:  
    commits = len(r.json()) if r.json() else 0
```

```
if commits < min_commits_count:  
    continue
```

```
url_pr = "https://api.github.com/search/issues"  
params_pr = {"q": f"author:{login}+repo:{repo_name}+type:pr+created:>={since}"}  
pr_response = requests.get(url_pr, headers=headers, params=params_pr, timeout=30)  
pr_count = pr_response.json().get("total_count", 0)
```

```
result.append({"login": login, "commits": commits, "prs": pr_count})
```

```
result.sort(key=lambda x: x["commits"], reverse=True)  
return result[:5]
```

```
def plot_activity(contributors, repo_name):  
    names = [c["login"] for c in contributors]
```

```
commits = [c["commits"] for c in contributors]
prs = [c["prs"] for c in contributors]
```

```
x = range(len(names))
width = 0.35
```

```
plt.figure(figsize=(10, 6))
plt.bar([i - width / 2 for i in x], commits, width, label="Коммиты", color="skyblue")
plt.bar([i + width / 2 for i in x], prs, width, label="Pull Requests", color="salmon")
plt.xlabel("Контрибьюторы")
plt.ylabel("Количество")
plt.title(f"Топ активных контрибьюторов в {repo_name}")
plt.xticks(x, names, rotation=45)
plt.legend()
plt.tight_layout()
plt.savefig(f'{repo_name.replace("/", "_")}activity.png')
plt.show()
```

```
if __name__ == "__main__":
    REPO_NAME = input("Введите репозиторий (owner/repo): ")
    DAYS_COUNT = int(input("Выберите период (7/30/365 дней): "))
    MIN_COMMITS_COUNT = int(input("Минимальное количество коммитов: "))

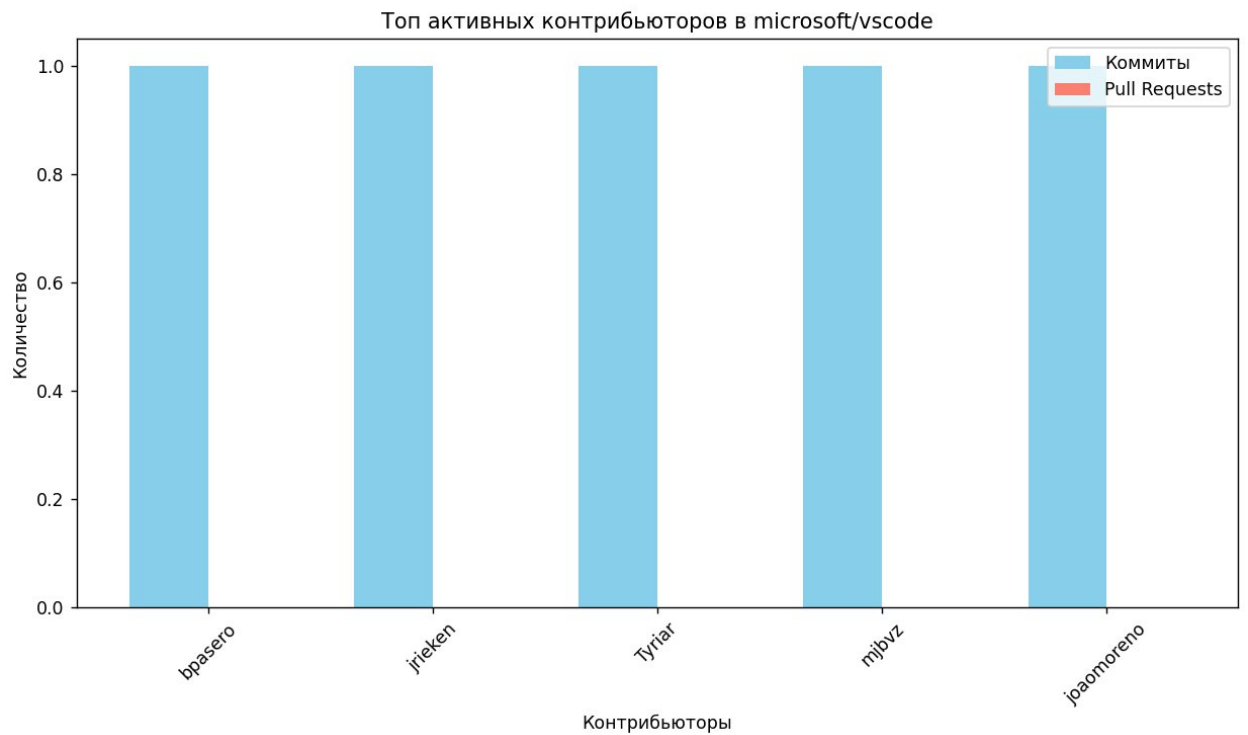
    TOP = get_top_contributors(REPO_NAME, DAYS_COUNT, MIN_COMMITS_COUNT)

    print(f"\nТОП-5 активных контрибьюторов в '{REPO_NAME}' за {DAYS_COUNT} дней:")
    for i, c in enumerate(TOP, 1):
        print(f"{i}. @{c['login']} - {c['commits']} коммитов, {c['prs']} PR")

    if TOP:
        plot_activity(TOP, REPO_NAME)
```

```
Введите репозиторий (owner/repo): microsoft/vscode
Выберите период (7/30/365 дней): 365
Минимальное количество коммитов: 1

ТОП-5 активных контрибьюторов в 'microsoft/vscode' за 365 дней:
1. @bpasero - 1 коммитов, 0 PR
2. @jrieken - 1 коммитов, 0 PR
3. @Tyriar - 1 коммитов, 0 PR
4. @mjbvz - 1 коммитов, 0 PR
5. @joaomoreno - 1 коммитов, 0 PR
```



Вывод: научился работать с Github API, приобрёл практические навыки написания программ для работы с REST API или GraphQL API